

Programmation objet en PHP

partie 2 : exceptions,
énumérations, encapsulation,
références

Gestion des erreurs, exceptions

- De façon similaire à d'autres langages, les erreurs d'exécution ou d'application en PHP sont gérées par un mécanisme d'*exceptions* et les instruction `try-catch-throw`
- Ces exceptions peuvent être attrapées (`catch`) pour réaliser un traitement de l'erreur correspondante.
- Les erreurs d'exécution (**Error**) sont lancées automatiquement par l'interprète, alors que les erreurs applicatives (**Exception**) sont lancées par l'instruction `throw`

Exceptions et erreurs

■ hiérarchie d'exceptions :

– **Error** implements Throwable

- ArithmeticError
 - » DivisionByZeroError
- AssertionError
- ParseError
- TypeError
 - » ArgumentCountError

La classe **Error** : les exceptions correspondant à une erreur d'exécution, lancées par l'interprète

– **Exception** implements Throwable

- *MyException*
- ...

La classe **Exception** : les exceptions correspondant à une erreur applicative, lancées par `throw`

```
interface Throwable extends Stringable {  
  
    public getMessage(): string  
    public getCode(): int  
    public getFile(): string  
    public getLine(): int  
    public getTrace(): array  
    public getTraceAsString(): string  
    public getPrevious(): ?Throwable  
    abstract public __toString(): string  
  
}
```

remarques

- Comme il n'y a pas de lien d'héritage entre `Error` et `Exception`, il est préférable de les *catcher* séparément (2 blocs catch),
- Une exception ou une error non catchée devient une erreur fatale et stoppe l'exécution de l'interprète,
- Il est d'usage dans un projet de sous-classer la classe `exception` pour catégoriser les erreurs,
- Les profils des fonctions et méthodes n'indiquent pas les exceptions/erreurs qu'elles sont susceptibles de lancer.

```
class ArgumentException extends Exception {};
```

```
function inverser (int|float $x): float {  
    if ($x === 0) throw  
        new ArgumentException('valeur invalide : 0');  
    return 1/$x;  
}
```

```
try {  
    print inverser(5);  
    print inverser(0);  
} catch (ArgumentException $e) {  
    print $e->getMessage();  
} catch (Error $e) {  
    print $e->getMessage();  
}
```

Recommandation (forte)

- Pour chaque cas d'erreurs lié à l'application, créer une classe exception dédiée pour catégoriser ces erreurs.
Eviter d'utiliser Exception

```
class ArgumentException extends Exception {};
```

```
class DatabaseConnectionException extends Exception {};
```

```
class DataNotFoundException extends Exception {};
```

```
class EmptyListException extends Exception {};
```

Enumérations : Enum

- Les énumérations permettent de définir un type qui est limité à une valeur parmi une liste de valeurs possibles.
- Utile pour typer toute propriété dont les valeurs possibles font partie d'une liste déterminée
 - État ou caractéristique d'un objet : (créée, confirmée, payée, livrée), (rouge, orange, vert, bleu), (XS, S, M, L, XL, XXL)
- Les valeurs peuvent être associées à une valeur scalaire (int, string)
- Peuvent porter des méthodes (y compris statiques) et implanter une interface

```
enum OrdreDeTri {  
    case Asc;  
    case Desc;  
}  
function query(string $field, OrdreDeTri $ordre= OrdreDeTri::Asc): array {}:  
query('nom', OrdreDeTri::Desc);
```

```
enum Status: int {  
    case CREE = 0;  
    case CONFIRME = 1;  
    case ANNULE = -1;  
    case CLOTURE = 10;  
  
    public function label(): string {  
        return match ($this) {  
            self::CREE=> 'Créé',      self::CONFIRME => 'Confirmé',  
            self::ANNULE => 'Annulé', self::CLOTURE => 'Cloturé'  
        };  
    }  
  
    public static function fromString(string $v): self {  
        return match($v) {  
            'CR'=> self::CREE,      'CO'=> self::CONFIRME,  
            'AN'=> self::ANNULE,    'CL'=> self::CLOTURE,  
            default=> self::CREE  
        };  
    }  
}
```

variables dynamiques

- La valeur d'une variable peut être utilisée comme un nom de variable ou d'attribut

```
$nom = 'michel';  
$variable = 'nom';  
  
echo "$variable : $nom : ${$variable}\n";  
$$variable = 'samia';  
echo "$variable : $nom : ${$variable}\n";
```

```
nom : michel : michel  
nom : samia : samia
```

```
$v = new Velo('rouge');  
$attribut = 'couleur';  
print "couleur du velo : {$v->$attribut} \n";
```

```
couleur du velo : rouge
```

- Ca marche aussi pour les fonctions/méthodes : toute variable suivie de () est interprétée comme un appel

```
$v = new Velo('rouge');  
  
$action = 'accelere';  
$vitesse = $v->$action(25);  
print "vitesse : $vitesse \n";  
  
$action = 'freine';  
$vitesse = $v->$action(10);  
print "vitesse : $vitesse \n";
```

```
vitesse : 25  
vitesse : 15
```

encapsulation et méthodes d'accès

- On peut utiliser ce principe pour programmer des getter génériques
- On conserve des setter spécifiques lorsqu'ils sont indispensables et pour implanter des contrôles / validations de valeurs
- On modifie les valeurs au travers de méthodes métier

example

```
class Velo implements Vehicule {  
  
    protected int $vitesse=0;  
    protected string $couleur;  
    protected bool $lumiere = false;  
  
    public function __construct(string $c) {  
        $this->couleur = $c;  
    }  
  
    public function get( string $attrname) : mixed {  
        if (property_exists($this, $attrname))  
            return $this->$attrname;  
        throw new Exception("$attrname : invalid property");  
    }  
  
    public function accelere(int $delta): int {  
        if ($delta > 0) $this->vitesse += $delta;  
        return $this->vitesse;  
    }  
  
    public function setLumiereOn() : void {  
        $this->lumiere = true;  
    }  
}
```

```
$velo = new Velo( "rouge" );  
$velo->accelere( 5 );  
echo $velo->get( "vitesse" );  
echo $velo->get( "couleur" );  
  
$velo->setLumiere0n( );
```

- un getter générique pour tous les attributs
- Des setter spécifiques pour les différents attributs

getter/setter magiques

- Certaines fonctions/méthodes PHP sont appelées automatiquement par l'interprète PHP dans certaines circonstances. Ces fonctions/méthodes sont dites "magiques".
- Lors d'un accès à un attribut d'un objet, si cet attribut est inaccessible (par exemple private) ou non déclaré, alors l'interprète va automatiquement transformer cet accès en un appel à la fonction `__get` / `__set`
- On peut programmer ces fonctions

```
public __get(string $name): mixed
```

- Autre version des getter/setter génériques, basée sur les méthodes magiques :

```
class Rectangle {  
    private int $la, $lo;  
  
    public function __construct(int $la,int $lo) {  
        $this->la = $la ;   $this->lo = $lo;  
    }  
  
    public function __get(string $at):mixed {  
        if (property_exists ($this, $at)) return $this->$at;  
        throw new Exception ("$at: invalid property");  
    }  
  
    public function setLargeur(int $la):void {  
        if ($la > 0) $this->la = $la;  
    }  
  
}
```

```
r = new Rectangle(3,4);  
$lo = $r->lo;  
  
$r->setLargeur(6);  
echo $r->la;
```

- l'encapsulation est réalisée en redéfinissant les accesseurs
- à l'utilisation : pas de getter + des setters spécialisés

- les méthodes magiques permettent des choses intéressantes :
 - accéder à des propriétés non visibles, ou n'existant pas
 - appeler des méthodes sous forme d'accès à une propriété

```
class Rectangle {  
    private int $la, $lo;  
  
    /* ... */  
  
    public function __get(string $at):mixed {  
        if ( property_exists ($this, $at) )  
            return $this->$at;  
        if ($at === 'width') return $this->la ;  
        if ($at === 'perimetre')  
            return ($this->largeur+$this->longueur)*2;  
        if ($at === 'surface')  
            return $this->computeSurface();  
        throw new Exception ("$at: invalid property");  
    }  
}
```

```
$r1 = new Rectangle ( 12, 24 ) ;
```

```
// appel automatique de la méthode __get  
// car la propriété n'est pas accessible
```

```
print $r1->width;  
print $r1->perimetre;  
print $r1->surface;
```

Références, affectation et paramètres

- Lors d'une affectation ou d'un passage de paramètre, la valeur est dupliquée pour les types simples et les tableaux

```
$a = $b ;
```

```
function foo(int $a): int {  
    return ++$a ;  
} ;  
foo($b);
```

\$a et \$b ont la même valeur mais
référencent 2 contenus différents

```
<?php  
$v1 = 5; $v2 = $v1;  
$v2 +=1 ;  
print "v1 : $v1 ; v2 : $v2";  
print "foo:".foo($v1). "v1: $v1";
```

```
v1 : 5 ; v2 : 6  
foo:6 v1:5
```

Manipulation de Références

- On peut affecter une *référence* : `&$v` désigne la référence à `$v` et non pas sa valeur

```
$a = & $b ;
```

```
function foo( & $a) {  
    return ++$a ;  
} ;  
foo($b);
```

`$a` et `$b` référencent le même contenu

```
<?php  
$v1 = 5; $v2 = & $v1;  
$v2 +=1 ;  
print "v1 : $v1 ; v2 : $v2";  
print "foo:".foo($v1). "v1: $v1";
```

```
v1 : 6 ; v2 : 6  
foo:7 v1:7
```

affectation d'objets, objets en paramètre

- Lorsque cela concerne des objets, l'affectation ou le passage de paramètres copient ***la référence***

```
<?php
$o1 = new Rectangle();
$o2 = $o1;

// $o1 et $o2 référencent le même objet

$o2->largeur = 123;
print $o1->largeur ;    // 123
```

Clonage d'objets

- On peut créer une copie d'un objet par clonage, ce qui permet par exemple d'affecter une copie d'un objet et non pas sa référence :

```
<?php
// affectation par référence
// $o1 et $o2 référencent le même objet
$o1 = new Rectangle();
$o2 = $o1;

// affectation avec clonage
// $o1 et $o3 sont deux objets différents
$o3 = clone $o1 ;
```